

Online Book Store Relational Database & Analytics

Comprehensive PostgreSQL Project Documentation & Query Portfolio

POSTGRESQL DATABASE PROJECT

Project Overview

This project demonstrates end-to-end relational database management and data analytics for an online bookstore using **PostgreSQL**. It encompasses table creation using PostgreSQL data types (such as SERIAL and NUMERIC), data definition modifications (DDL), relational foreign key constraints, and analytical SQL queries ranging from basic data retrieval to advanced multi-table JOINS and aggregations.

1. PostgreSQL Database Schema Design

The system consists of three main entities connected via foreign key constraints:

Table Name	Primary Key	Foreign Keys	Description
Books	Book_ID	None	Stores book catalog information including title, author, genre, price, and current stock level.
Customers	Customer_ID	None	Stores customer contact profiles, email addresses, and location data.
Orders	Order_ID	Customer_ID, Book_ID	Tracks sales transactions, order dates, quantities ordered, and total order amounts.

PostgreSQL Table Schema Definitions (DDL)

Creating Books Table

```
DROP TABLE IF EXISTS Books;
CREATE TABLE Books(
    Book_ID SERIAL PRIMARY KEY,
    Title VARCHAR(255),
    Author VARCHAR(50),
    Genre VARCHAR(50),
    Published_year INT,
    Price NUMERIC(10,2),
    Stock INT
);
```

Creating Customers Table

```
DROP TABLE IF EXISTS Customers;
CREATE TABLE Customers(
  Customer_id SERIAL PRIMARY KEY,
  Name VARCHAR(100),
  Email VARCHAR(50),
  Phone INT,
  City VARCHAR(50),
  Country VARCHAR(255)
);
```

Creating Orders Table

```
DROP TABLE IF EXISTS Orders;
CREATE TABLE Orders(
  Order_ID SERIAL PRIMARY KEY,
  Customer_ID INT REFERENCES Customers(Customer_ID),
  Book_ID INT REFERENCES Books(Book_ID),
  Order_date DATE,
  Quantity INT,
  Total_Amount NUMERIC(10,2)
);
```

2. Basic SQL Queries & Data Filtering

Q1. Retrieve all books in the 'Fiction' genre:

```
SELECT * FROM Books
WHERE genre = 'Fiction';
```

Q2. Find books published after the year 1950:

```
SELECT * FROM Books
WHERE published_year > 1950;
```

Q3. List all customers from Canada:

```
SELECT * FROM Customers
WHERE Country = 'Canada';
```

Q4. Show orders placed in November 2023:

```
SELECT * FROM Orders
WHERE order_date BETWEEN '2023-11-01' AND '2023-11-30';
```

Q5. Retrieve the total stock of books available:

```
SELECT SUM(Stock) AS Total_Stock
FROM Books;
```

Q6. Find details of the most expensive book:

```
SELECT * FROM Books
ORDER BY Price DESC
LIMIT 1;
```

Q7. Show all customers who ordered more than 1 quantity of a book:

```
SELECT * FROM Orders
WHERE quantity > 1;
```

Q8. Retrieve all orders where the total amount exceeds \$20:

```
SELECT * FROM Orders
WHERE total_amount > 20;
```

Q9. List all unique genres available:

```
SELECT DISTINCT genre FROM Books;
```

Q10. Find the book with the lowest stock:

```
SELECT * FROM Books
ORDER BY stock
LIMIT 1;
```

Q11. Calculate total revenue generated across all orders:

```
SELECT SUM(Total_amount) AS total_revenue
FROM orders;
```

3. Advanced PostgreSQL Analytics & Business Insights

Q1. Total number of books sold per genre:

```
SELECT b.genre, SUM(o.Quantity) AS total_book_sold
FROM Orders o
JOIN Books b ON o.book_id = b.book_id
GROUP BY b.Genre;
```

Q2. Average price of books in the 'Fantasy' genre:

```
SELECT AVG(price) AS Average_price
FROM Books
WHERE Genre = 'Fantasy';
```

Q3. Customers who have placed at least 2 orders:

```
SELECT o.customer_id, c.name, COUNT(o.order_id) AS order_count
FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id
GROUP BY o.customer_id, c.name
HAVING COUNT(o.order_id) >= 2;
```

Q4. Most frequently ordered book:

```
SELECT o.book_id, b.title, COUNT(o.order_id) AS Order_count
FROM orders o
JOIN books b ON o.book_id = b.book_id
GROUP BY o.book_id, b.title
ORDER BY Order_count DESC
LIMIT 1;
```

Q5. Top 3 most expensive 'Fantasy' books:

```
SELECT * FROM Books
WHERE genre = 'Fantasy'
ORDER BY price DESC
LIMIT 3;
```

Q6. Total quantity of books sold by author:

```
SELECT b.author, SUM(o.quantity) AS total_book_sold
FROM orders o
JOIN books b ON o.book_id = b.book_id
GROUP BY b.author;
```

Q7. Cities where customers spent over \$30:

```
SELECT DISTINCT c.city, o.total_amount
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.total_amount > 30;
```

Q8. Customer with the highest overall spend:

```
SELECT c.customer_id, c.name, SUM(o.total_amount) AS total_spent
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name
ORDER BY total_spent DESC
LIMIT 1;
```

Q9. Calculate remaining stock after fulfilling all orders:

```
SELECT b.book_id, b.title, b.stock,
       COALESCE(SUM(o.quantity), 0) AS order_quantity,
       b.stock - COALESCE(SUM(o.quantity), 0) AS Remaining_quantity
FROM books b
LEFT JOIN orders o ON b.book_id = o.book_id
GROUP BY b.book_id, b.title, b.stock
ORDER BY b.book_id;
```